

Lab0 - Course Onboarding & Prework: Intermediate Web Development (WEB102)

Name: Yumin Jang

FAU Z Number: Z23655899

FAU Email: yjang2022@fau.edu

GitHub Username: Ghostcomputer453145

GitHub Prework Repository Link:

https://github.com/Ghostcomputer453145/web102_prework

Lab 0 Github intermediate workshop link:

<https://github.com/FAU-FullStack-Dev-Spring2026/lab0-github-intermediate-workshop-Ghostcomputer453145>

Course: COP 4808-001 13815 Full-Stack Web Development

Professor: Dr. David Jaramillo

Date: 1/18/2026

Part 1 – CodePath Registration

Screenshot of the complete registration of CodePath:

CODEPATH*ORG

Hi Yumin,

 **Congratulations!** Your admission for the upcoming Intermediate Web Development course at Florida Atlantic University has been verified and you now have access to the course materials.

To ensure smooth access, log in to your registered GitHub account, Ghostcomputer453145, (in the GitHub platform) first before clicking the button below.

 **Access your Course Portal**

 **Course Portal**

This is CodePath's course delivery platform, where you will:

 **Course Portal**

This is CodePath's course delivery platform, where you will:

- Access your lecture slides and lab for each unit
- Submit your project assignments
- View your grades

CodePath Benefits

All students who complete this course receive special benefits:

- Receive a (digital) CodePath certificate of completion.
- Are CodePath Alumni and gain access to alumni networks.
- Gain access to the [Student Hub](#).

Have questions about grading? Email support@codepath.org. For assignment help, you can reach out to dedicated technical support reps on [Slack](#).

We're excited to have you join us!

Best regards,

The Team at CodePath

Application Status

You have applied to the **For Credit - Intermediate Web Development** program.

Thanks for applying!

You may expect to receive an update within 24 hours. For questions or concerns, please email admissions@codpath.org.

Initial Application: **Submitted**

Submitted on January 11

[VIEW](#) / [EDIT](#) / [CANCEL](#)

Share your referral link

<https://apply.codepath.org/cohorts/fc-web102-sp26/versi>

Copy link 

or just share your referral code

wKp0lDvj

Copy 

0 applicants have applied using your code.

Schedule: The class will follow along the schedule arranged by their university/college.

Add us to your contacts

To make sure you receive all CodePath emails, please add our emails to your trusted list of senders, contacts or address book

- support@codepath.org
- admissions@codepath.org

Your providers do the best they can to keep spam out, but sometimes the systems they use mistakenly catch good mail along with it.

If you have any questions, reach out to us at admissions@codepath.org.

CodePath Courses

Courses ▾

🔗 Quick Links ▾

⚡ Quick Actions ▾

Yumin Jang ▾

CODE
PATH
+ORG

WEB102 | Intermediate Web Development

For-Credit Intermediate Web Development Spring 2026 (@ Florida Atlantic University)
Personal Member ID#: 112692

Need help? Post on our [Class Slack Channel](#) or email us at admissions@codepath.org

Getting Started

Learning with AI ✨

► Course Info

★ Course Progress

Unit 1

Unit 2

Unit 3

Unit 4

Unit 5

Unit 6

Unit 7

Unit 8

🏠 Overview

Lab

Projects

✨ IDE Setup

✨ Learning with AI ✨

📁 Resources

📊 Report

🔔 REMINDER: Submit your [Unit 1 Project](#) by **Sunday, February 1st at 11:59PM EST** using the **Submit** button on the [Project tab](#)

Unit 1: Diving Into React

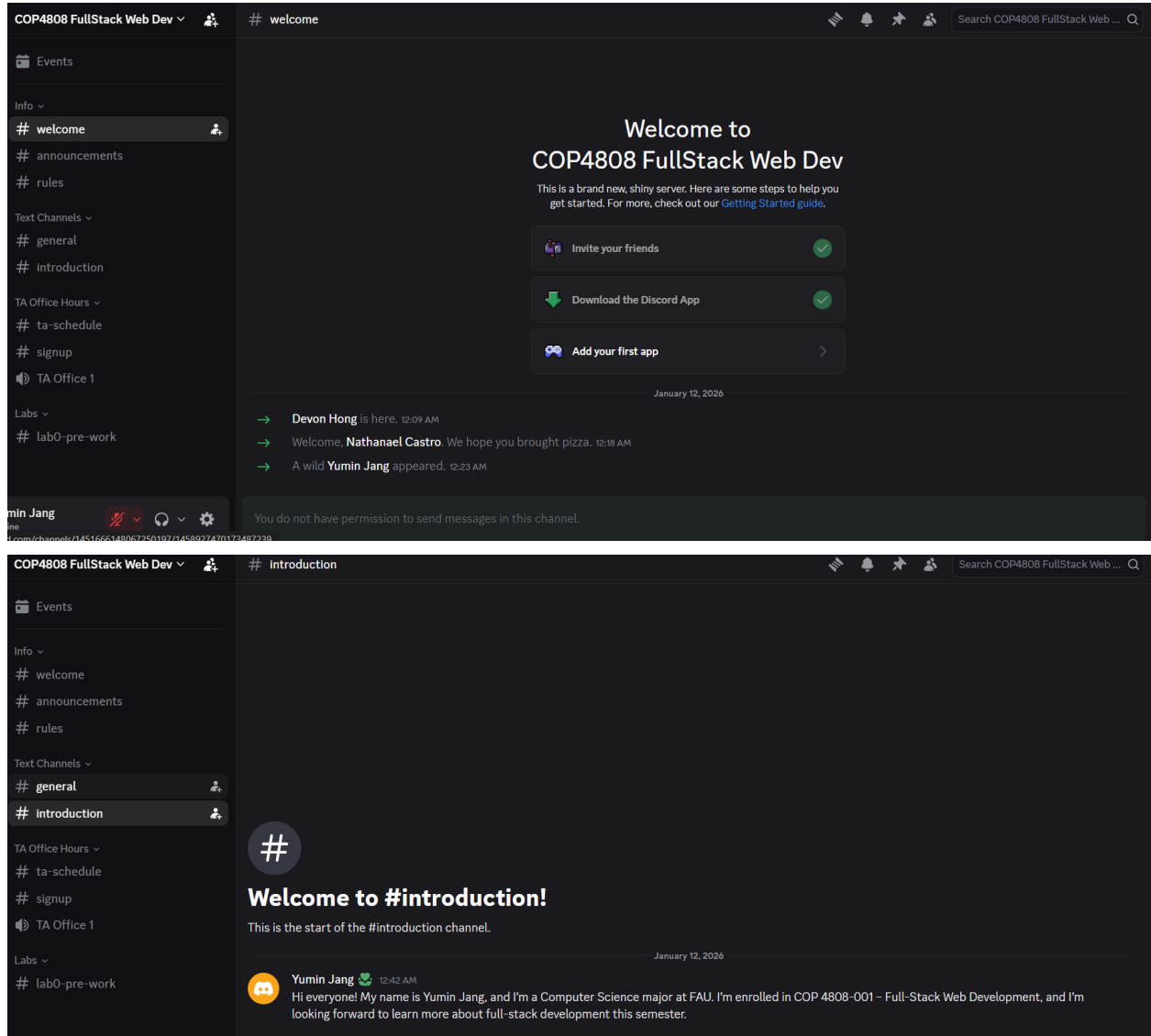
Welcome to Unit 1! We are so glad to have you.

This unit is all about [React](#), a frontend web framework created by Facebook that allows for the creation of decompose-able, reusable user interfaces. What do we mean by all these terms?

- Frontend web framework: When we say frontend, we mean creating what the user interacts with. We think you'll find that React makes your life easier. Your CSS will look roughly the same as before, but now you'll be combining HTML and JavaScript together into JSX. Now, in addition to stuff like `<p>` and `<div>`, you'll start creating your own elements (called components) like `<BanListPane>`, `<Card>`, and `<Post>`.
- Decompose-able: React is all about components. Take a look at a site like google.com and you'll see the same section of the UI (user interface) repeated over and over: a search result. When using React, we'll break websites down into parts and tackle them one at a time.
- Reusable: Just like Legos, you can use components in a lot of different ways. Build a castle, a car, or a life-size zebra, all using the

Part 2 – Discord Registration

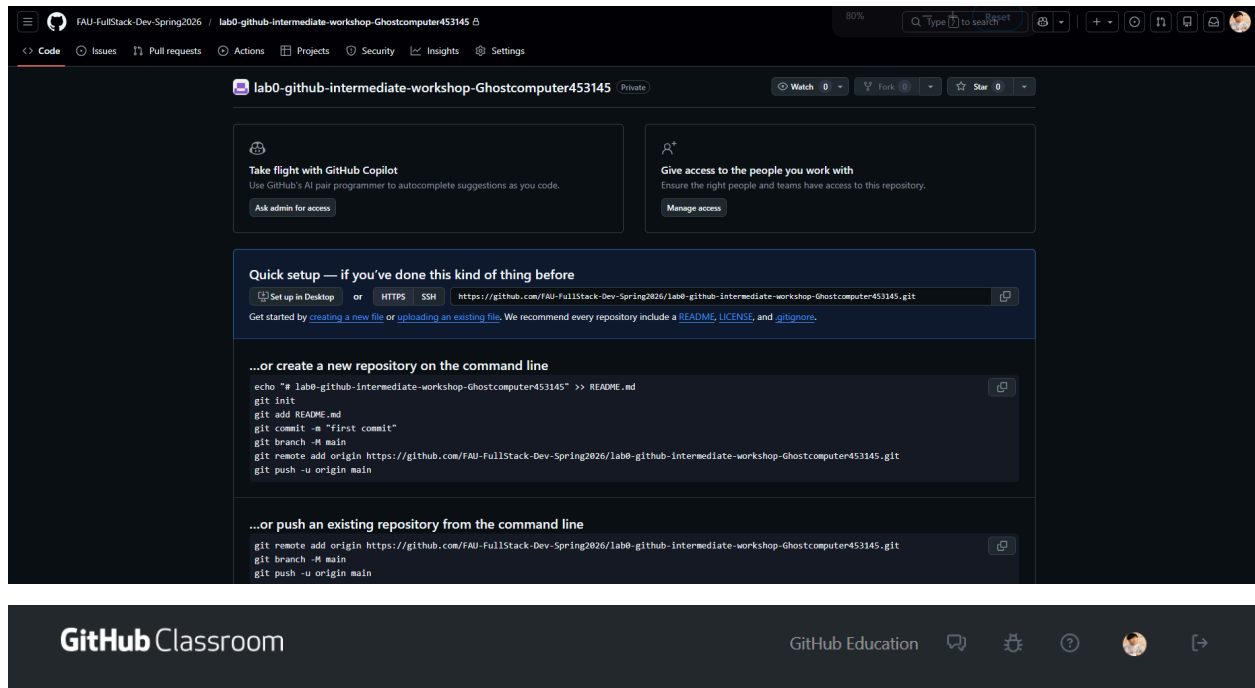
Screenshot of my completion of registration & introduction on discord:



Part 3 – GitHub Classroom Repository

Screenshot of the GitHub Classroom Repository:

Before:



You're ready to go!

You accepted the assignment, **lab0-github-intermediate-workshop**.

Your assignment repository has been created:

 <https://github.com/FAU-FullStack-Dev-Spring2026/lab0-github-intermediate-workshop-Ghostcomputer453145>

We've configured the repository associated with this assignment.

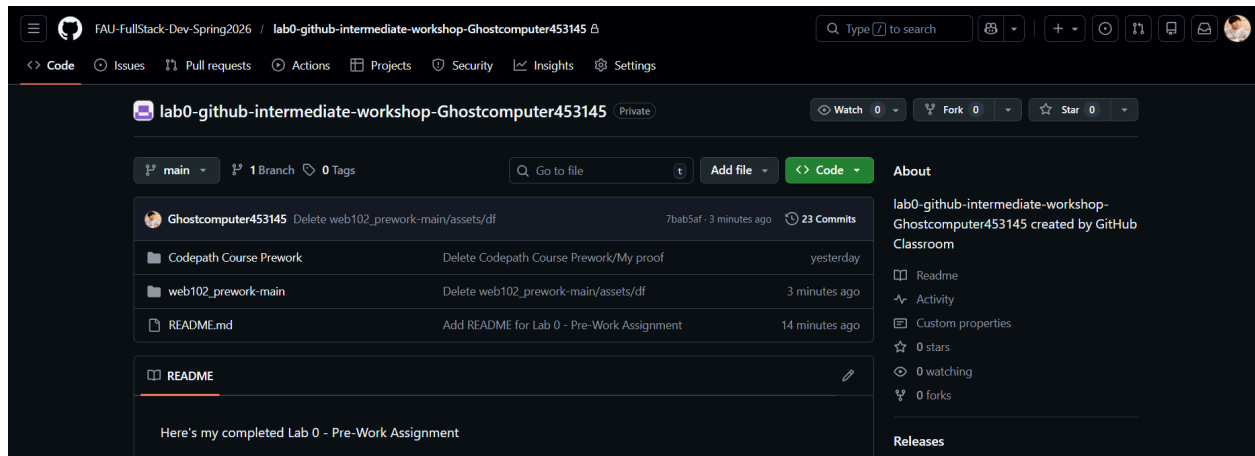


Join the GitHub Student Developer Pack

Verified students receive free GitHub Pro plus thousands of dollars worth of the best real-world tools and training from GitHub Education partners — for free. For more information, visit [GitHub Student Developer Pack](https://github.com/education).

[Apply](#)

After:



Part 4 – CodePath Course Prework

Completed Challenges

- ☒ Challenge-1
- ☒ Challenge-2
- ☒ Challenge-3
- ☒ Challenge-4
- ☒ Challenge-5
- ☒ Challenge-6
- ☒ Challenge-7

Challenge 0-7 Secret Keys

Challenge 0 secret key: seaworthy

Challenge 1 secret key: OOZEdiverTRAPpine

Challenge 2 secret key: 6games-container.stats-card15

Challenge 3 secret key: 11seafoamGAMES_JSON

Challenge 4 secret key: 19187800268BRAIN

Challenge 5 secret key: 74FLANNELclick

Challenge 6 secret key: toLocaleString<div>1IVY

Challenge 7 secret key: ZooHowCEDAR

Challenge Documentation

Challenge 0

Description:

This challenge introduced the CodePath WEB102 Prework and explained how the challenge system works. It required reading through the instructions, understanding the tools and topics that will be covered (HTML, CSS, JavaScript, Git, and React preparation), and learning how to unlock each challenge using secret keys. The goal was to become familiar with the overall structure of the prework before beginning any coding tasks.

Code / Work Shown: (There was no work to be done or shown just inserting the secret key to the next challenge)

Tips for The Pework

- Record each  **Secret Key** as you discover it. That way, you can return to the challenge you are currently working on later.
- Go through the extra resources! We've included some helpful guides and explanations if you're curious and want to learn more.
- Notes about  **Secret Keys:**
 - Important: Keys for the Offline PDF version of prework **are case sensitive!**
 - (For example, "SEA_MONSTER" is NOT the same as "sea_monster")
 - Secret keys can be any combination of letters, numbers, and symbols.

Ready?

Your first secret key is **seaworthy**. Go!

CODEPATH*ORG

WEB102 Pework

Challenge 1 of 7: Getting Started

Background

About Git

Throughout this course, you will be using Git to keep track of your work, prepare assignments for submission, and collaborate with others. Git is a version control system that allows programmers to save versions of their work over time, much like the version history in Google Suite products. Git also allows multiple people to collaborate on the same code without overwriting each others' work. The system will keep track of changes and allow you to merge them together in an organized way.

Using Git to Keep Track of Changes

At its core, Git keeps track of changes to files and reconciles differences between different versions of the same file. For example, you just forked a repository created by CodePath. A repository is simply a location where code is stored. Forking created your own copy of the code separate from the one that CodePath owns. Git is complex and it will take time to fully understand. If some of the commands below seem obscure, just know that it feels that way to everyone at first and over time they will start to make more sense.

It's common to interact with Git using the terminal or command prompt. If this is your first time using the terminal, it can seem overwhelming! The terminal is a way for developers to communicate with the operating system. Throughout this course, you'll become more comfortable as you continue to practice with this environment. For Git specifically, commands should start with `git`, followed by one of the operations Git can perform.

What I Learned:

I learned how the WEB102 prework is structured and how secret keys are used to unlock each challenge. I also gained a clear overview of the tools and JavaScript concepts that will be essential for building React applications later in the course.

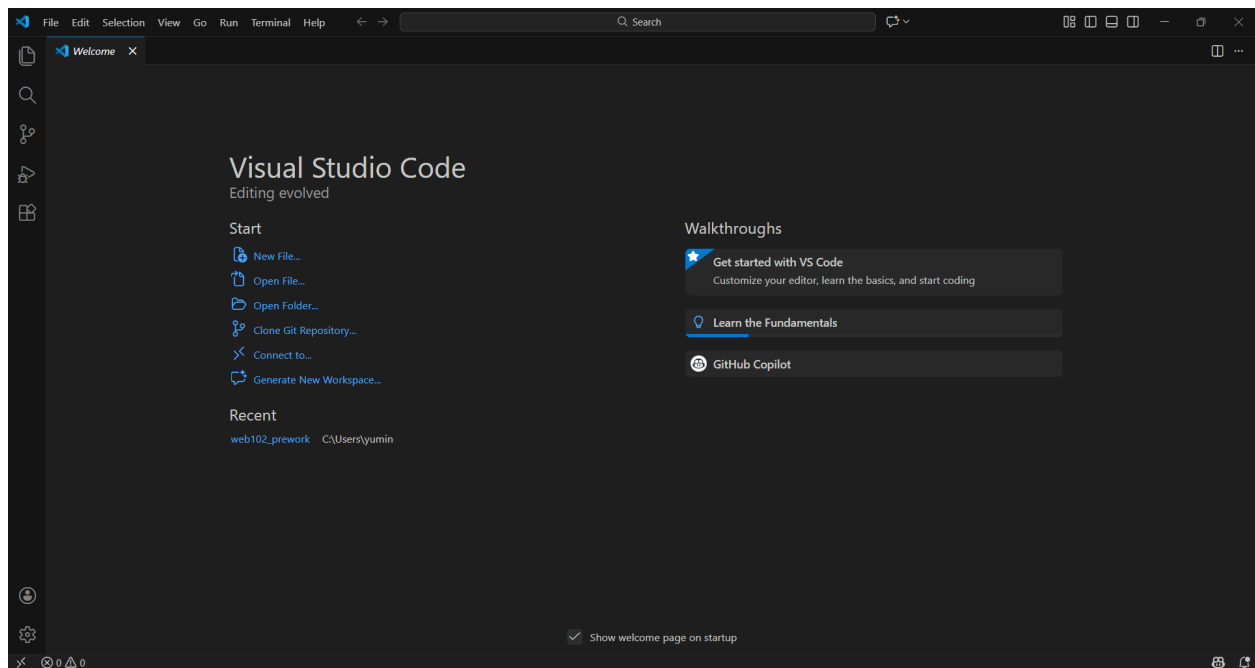
Challenge 1

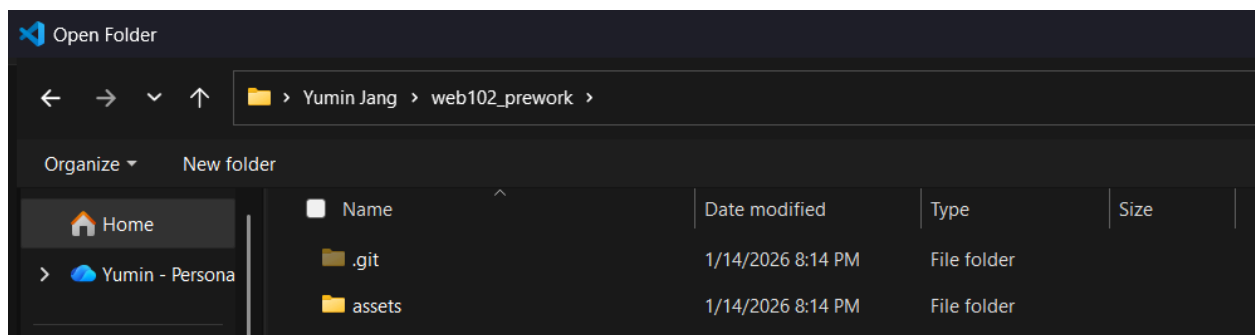
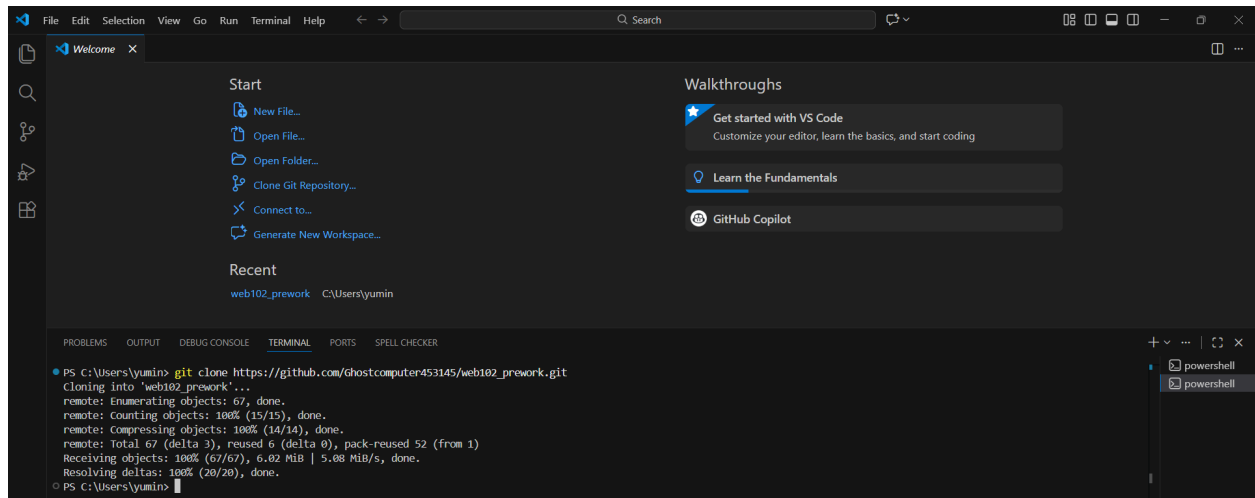
Description:

This challenge focused on setting up the development environment and learning the fundamentals of Git and GitHub. It required installing essential tools such as Visual Studio Code and Git, creating and signing into a GitHub account, forking the WEB102 prework repository, and cloning it locally using Git commands. The challenge also introduced core Git concepts such as repositories, staging, committing, branching, and syncing code between local and remote repositories. Finally, it tested understanding through a Git quiz to unlock the next challenge.

Code / Work Shown:

Here's the VS Code installed and opened as well as the terminal showing the successful git clone command and repository files listed.





Here's the completed Git quiz and successful entry of the secret key to unlock Challenge 2

1. Which command starts a fresh new repository?

1. `git clone <link-to-repository>` (keyword: SKEW)
2. `git init` (keyword: OOZE)
3. `git push origin master` (keyword: ALLY)
4. `git --version` (keyword: FLAP)

2. What happens when executing the command `git add index.js`?

1. A copy of the file `index.js` is created in the working directory (keyword: crib)
2. A copy of the file `index.js` is added to the remote repo (keyword: harp)
3. The file `index.js` is moved from the local repo to the staging area (keyword: best)
4. The file `index.js` is moved from the working directory to the staging area (keyword: dive)

3. What is the process for adding files to the local repository called?

1. committing (keyword: TRAP)
2. pushing (keyword: LIME)
3. forking (keyword: FUSE)
4. cloning (keyword: GAZE)

4. What happens when executing the command `git branch bug-fixes` then `git checkout bug-fixes`?

1. Code from the branch `bug-fixes` will be pushed to the remote repository. (keyword: zest)
2. All commits on the branch `bug-fixes` will be listed in the terminal. (keyword: walk)
3. A new branch `bug-fixes` will be created and then moved to. (keyword: pine)
4. The branch `bug-fixes` will be updated with all recent commits. (keyword: play)

1. OOZE

2. dive

3. TRAP

4. pine

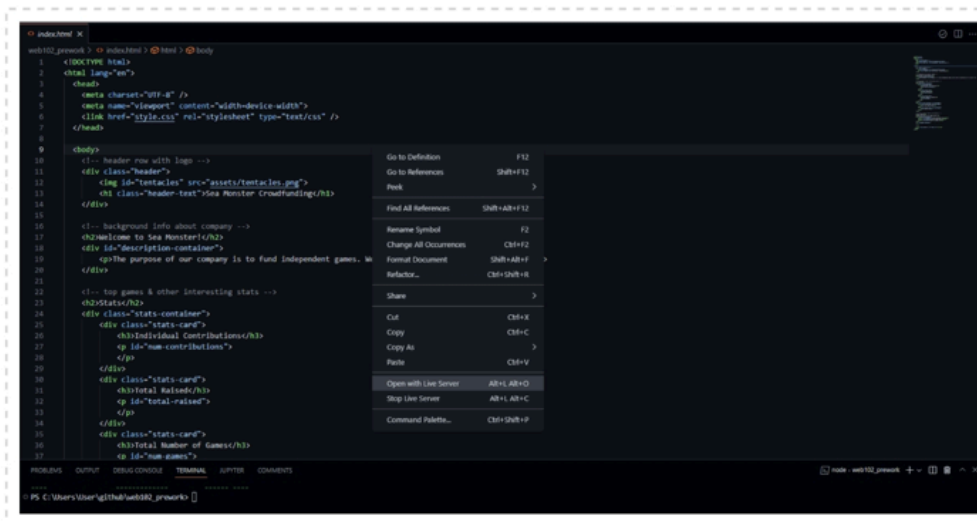
Secret key: OOZEdivetrAPP

Challenge 2 of 7: Review the Starter Code

✓ Checkpoint

Great work on that Git quiz! Are you feeling more confident about using Git for your coding projects? In this challenge, you'll check out the starter code we've provided you with in order to set yourself up for the upcoming challenges.

You can run the starter code to view the website using the recommended Live Server plugin by right-clicking within the `index.html` file and then selecting "Open with Live Server". Here's a screenshot showing how to start the Live Server:



At this point, your site should look like the exemplar below - you won't have made any changes yet, but you will in just a moment!

Moving on to Challenge 3

Create the  **Secret Key** from its components with no spaces in between:

- Component 1: How many attributes does each game have?
- Component 2: What is the id of the HTML element that will display a list of all games?
- Component 3: Which selector would select all elements with the stats-card class? Include the full selector (in other words, what would you write if defining a CSS rule for these elements?).
- Component 4: Which line number in index.js contains the start of a loop?

Hint: your  **Secret Key** should be 29 characters long.

Reminders about  **Secret Keys:**

- Keys for the Offline PDF version of prework **are case sensitive!**
 - (For example, "SEA_MONSTER" is NOT the same as "sea_monster")
- Secret keys can be any combination of letters, numbers, and symbols.

Use your  **Secret Key** to unlock the next PDF file. If your  **Secret Key** is correct, you will move onto the next step!

What I Learned:

I learned how to use Git and GitHub to manage code versions, clone repositories, and understand how changes move between the working directory, staging area, and repository. This challenge helped me become more comfortable using the terminal and basic Git commands that will be used throughout the course.

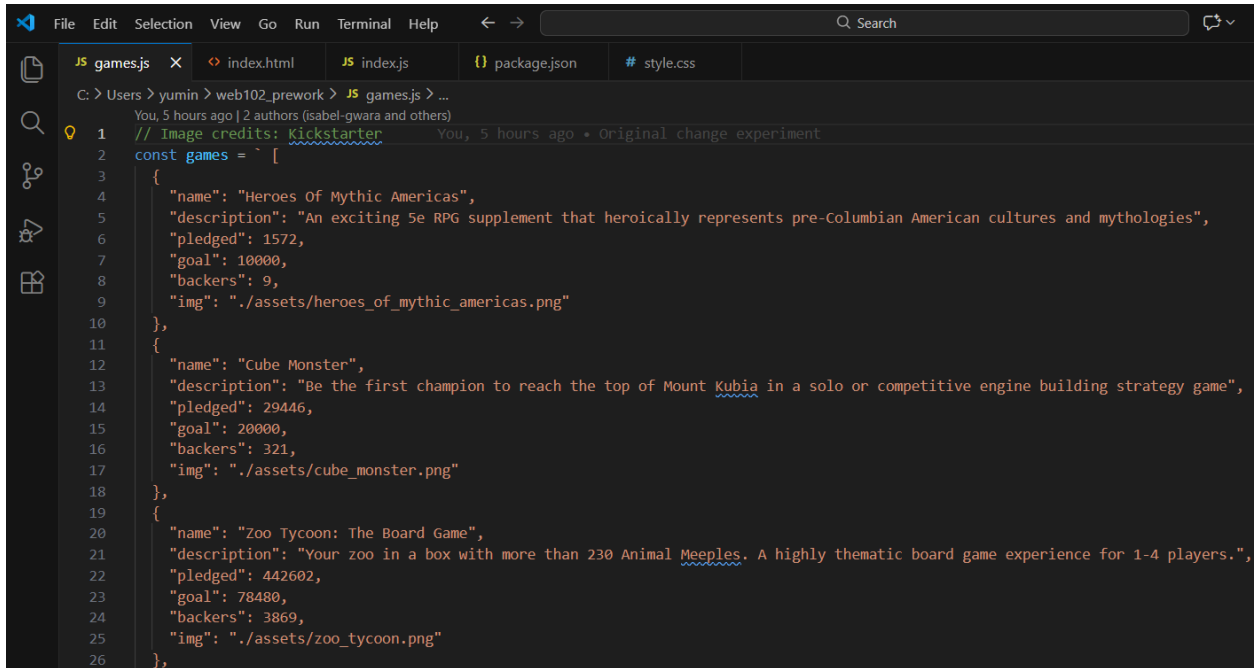
Challenge 2

Description:

This challenge focused on reviewing and understanding the starter code provided for the WEB102 prework project. It required examining multiple files in the repository, including games.js, index.html, style.css, and index.js. The goal was to become familiar with the structure of the project, understand how game data is stored using JSON, identify key HTML elements used to display content, and apply basic CSS styling using selectors and flexbox. The challenge also reinforced reading and interpreting existing JavaScript code before making changes.

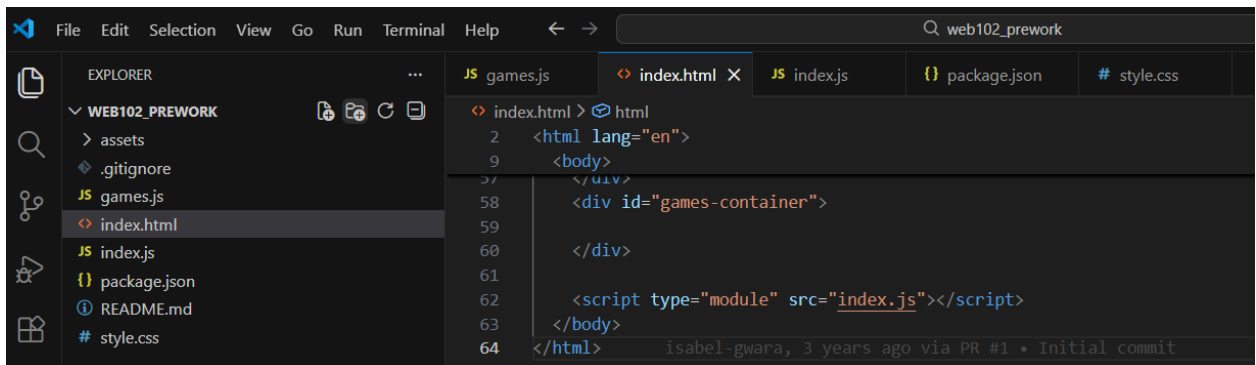
Code / Work Shown:

In Secret Key component 1 there are 6 properties in each game which are name, description, pledged, goal, backers, and img.

A screenshot of the Visual Studio Code editor. The 'EXPLORER' sidebar on the left shows the file structure of a project named 'web102_prework', with files like 'assets', '.gitignore', 'games.js', 'index.html', 'index.js', 'package.json', 'README.md', and 'style.css'. The 'index.html' file is selected. The main editor area shows the content of 'games.js'. It starts with a comment about image credits from Kickstarter. Then, a JavaScript array named 'games' is defined, containing three game objects. Each object has properties for 'name', 'description', 'pledged', 'goal', 'backers', and 'img'. The games listed are 'Heroes Of Mythic Americas', 'Cube Monster', and 'Zoo Tycoon: The Board Game'.

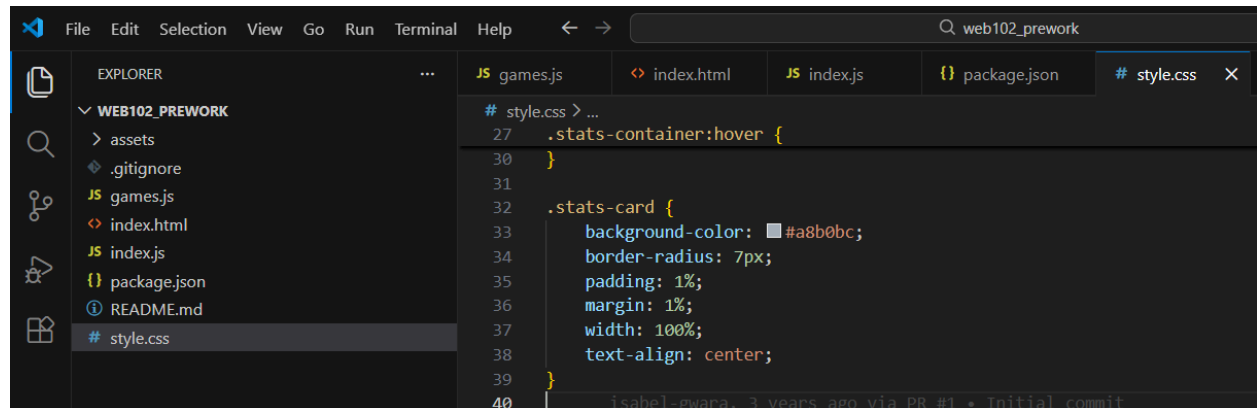
```
1 // Image credits: Kickstarter
2 const games = [
3   {
4     "name": "Heroes Of Mythic Americas",
5     "description": "An exciting 5e RPG supplement that heroically represents pre-Columbian American cultures and mythologies",
6     "pledged": 1572,
7     "goal": 10000,
8     "backers": 9,
9     "img": "./assets/heroes_of_mythic_americas.png"
10  },
11  {
12    "name": "Cube Monster",
13    "description": "Be the first champion to reach the top of Mount Kubia in a solo or competitive engine building strategy game",
14    "pledged": 29446,
15    "goal": 20000,
16    "backers": 321,
17    "img": "./assets/cube_monster.png"
18  },
19  {
20    "name": "Zoo Tycoon: The Board Game",
21    "description": "Your zoo in a box with more than 230 Animal Meeples. A highly thematic board game experience for 1-4 players.",
22    "pledged": 442602,
23    "goal": 78480,
24    "backers": 3869,
25    "img": "./assets/zoo_tycoon.png"
26  },
27 ]
```

In Secret Key component 2 the id of the HTML element that will display a list of all games is games-container.

A screenshot of the Visual Studio Code editor showing the 'index.html' file. The 'EXPLORER' sidebar on the left shows the same file structure as the previous image. The main editor area shows the HTML code for 'index.html'. It includes a basic HTML structure with a 'body' tag. Inside the body, there is a 'div' element with the id 'games-container'. At the bottom, there is a script tag that loads 'index.js' as a module.

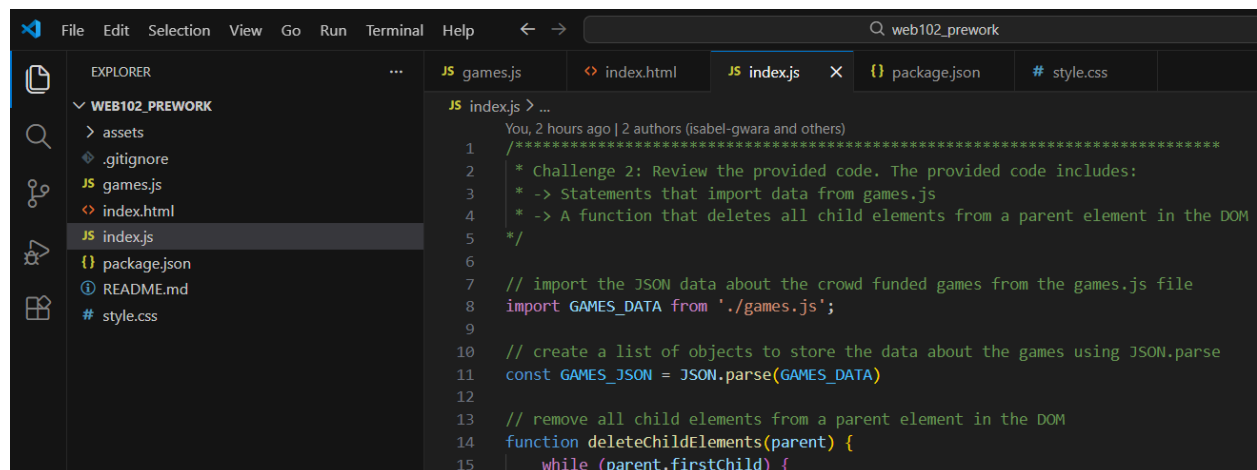
```
2 <html lang="en">
9 <body>
10 </div>
58 <div id="games-container">
59
60 </div>
61
62 <script type="module" src="index.js"></script>
63 </body>
64 </html>
```

In Secret Key component 3 the selector that select all elements with the stats-card class is .stats-card.



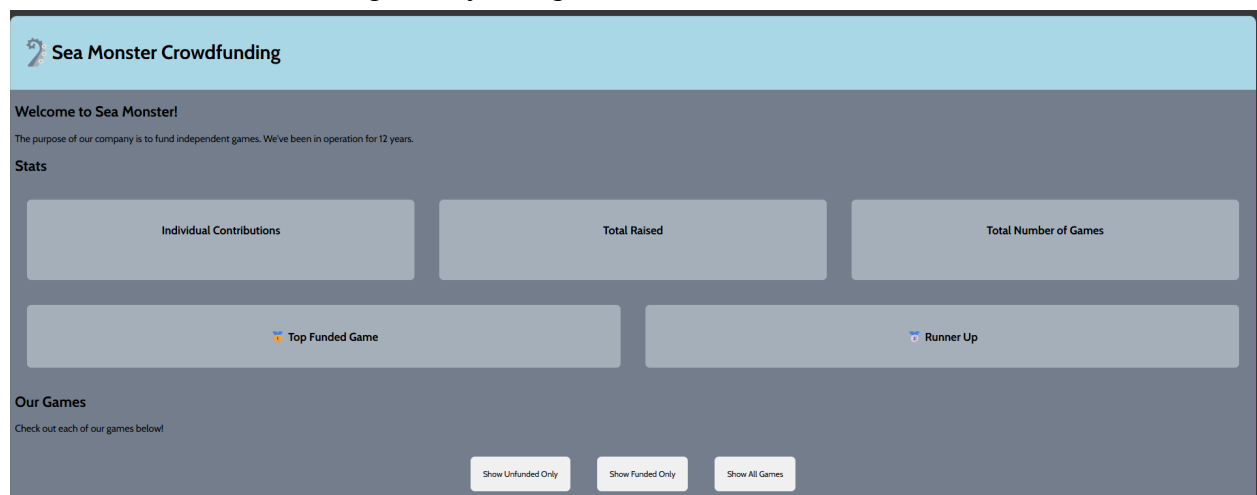
```
# style.css > ...
27 .stats-container: hover {
28
29 }
30
31
32 .stats-card {
33   background-color: #a8b0bc;
34   border-radius: 7px;
35   padding: 1%;
36   margin: 1%;
37   width: 100%;
38   text-align: center;
39 }
40
```

In Secret Key component 4 the line number in index.js contains the start of a loop is 15.



```
JS index.js > ...
You, 2 hours ago | 2 authors (isabel-gwara and others)
1 / *****
2 * Challenge 2: Review the provided code. The provided code includes:
3 * -> Statements that import data from games.js
4 * -> A function that deletes all child elements from a parent element in the DOM
5 */
6
7 // import the JSON data about the crowd funded games from the games.js file
8 import GAMES_DATA from './games.js';
9
10 // create a list of objects to store the data about the games using JSON.parse
11 const GAMES_JSON = JSON.parse(GAMES_DATA)
12
13 // remove all child elements from a parent element in the DOM
14 function deleteChildElements(parent) {
15   while (parent.firstChild) {
```

Here's the website running locally using Live Server



What I Learned:

I learned how to analyze starter code by identifying how data, HTML structure, CSS styling, and JavaScript logic work together. This challenge helped me feel more comfortable navigating a multi-file project and understanding existing code before modifying it. With all of the components together I got this secret key:

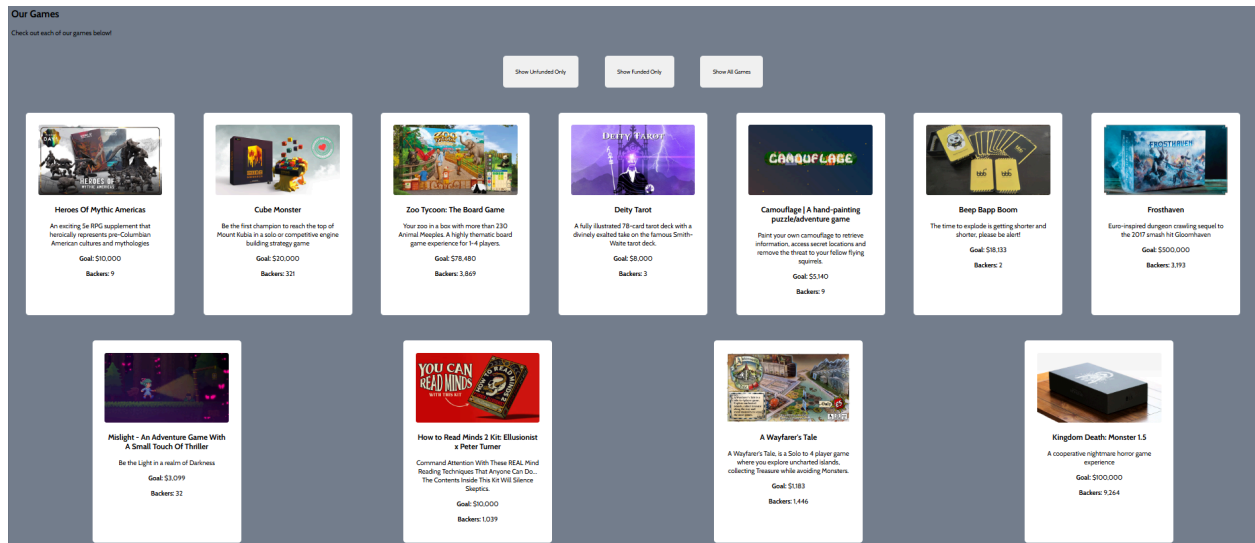
6games-container.stats-card15

Challenge 3**Description:**

This challenge required dynamically displaying game data on the webpage by creating a card for each game using JavaScript. It focused on working with the DOM, looping through an array of game objects, and generating HTML elements programmatically. Using a function, a loop, template literals, and DOM manipulation methods, each game's image and details were added as a styled card under the "Our Games" section. This challenge reinforced how JavaScript can be used to create and modify webpage content based on data.

Code / Work Shown:

Here's a screenshot of multiple game cards displayed with images, titles, and text visible in the "Our Games" section.



```

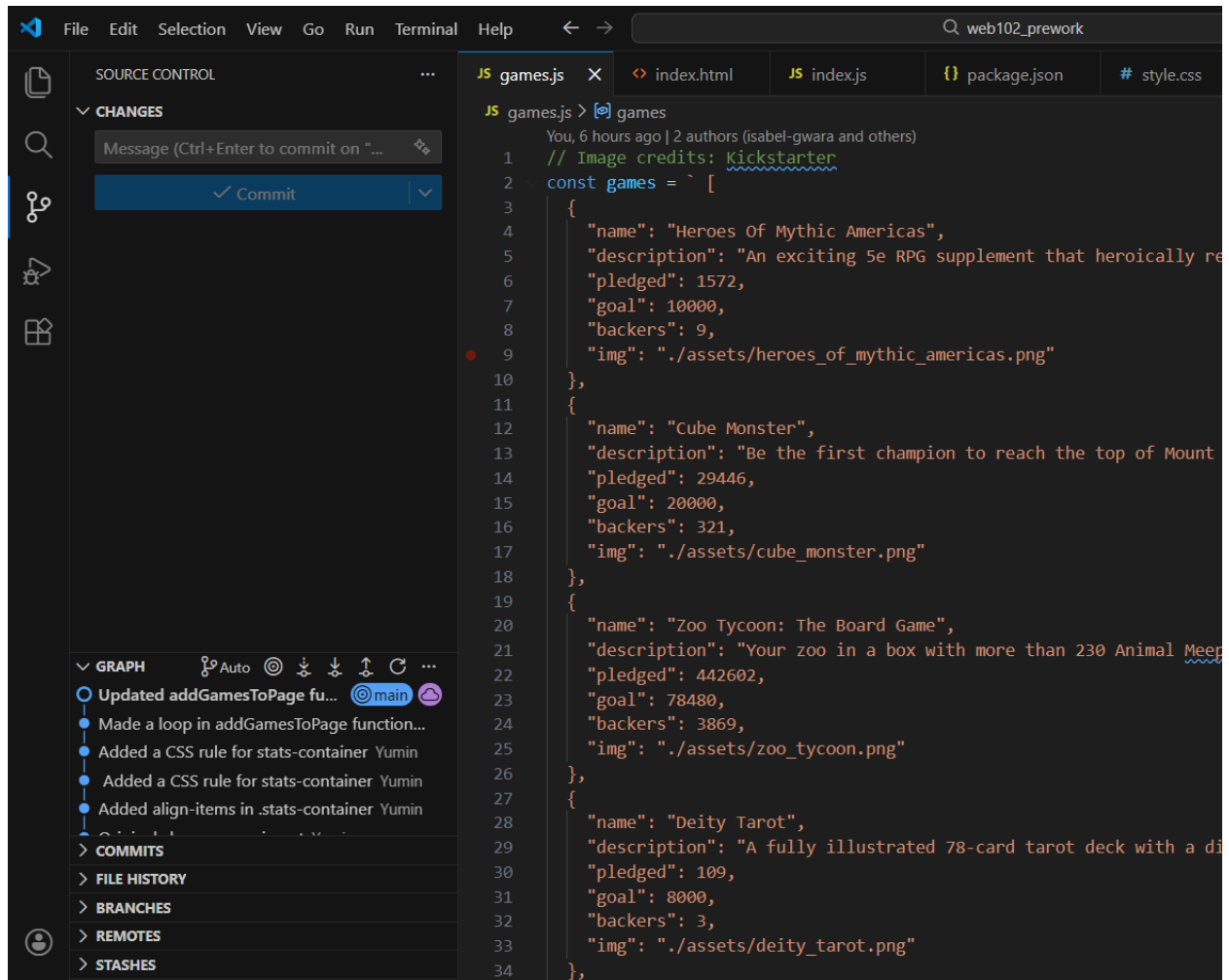
File Edit Selection View Go Run Terminal Help
web102_prework-main

EXPLORER
WEB102_PREWORK-MAIN
  assets
  CodePath Challenges
  .gitignore
  games.js
  index.html
  JS index.js
  Lab 0 - Pre-Work Assignment.pdf
  package.json
  README.md
  SeaMonsterCrowdfunding.gif
  style.css

JS index.js
19
20
21 // *****
22 * Challenge 3: Add data about each game as a card to the games-container
23 * Skills used: DOM manipulation, for loops, template literals, functions
24 */
25
26 // grab the element with the id games-container
27 const gamesContainer = document.getElementById("games-container");
28
29 // create a function that adds all data from the games array to the page
30 function addGamesToPage(games) {
31   for (let i = 0; i < games.length; i++) {
32     const game = games[i];
33
34     const gameCard = document.createElement("div");
35     gameCard.classList.add("game-card");
36
37     gameCard.innerHTML = `
38       
39       <h3>${game.name}</h3>
40       <p>${game.description}</p>
41       <p><strong>Goal:</strong> ${game.goal.toLocaleString()}</p>
42       <p><strong>Backers:</strong> ${game.backers.toLocaleString()}</p>
43     `;
44     gamesContainer.appendChild(gameCard);
45   }
46 }
47
48 addGamesToPage(GAMES_JSON);
49

```

For Secret Key component 1 the loop will run 11 times when called with GAMES_JSON since there's 11 games.



```
2 const games = [
36   {
37     "name": "Camouflage | A hand-painting puzzle/adventure game",
38     "description": "Paint your own camouflage to retrieve information, access secret location",
39     "pledged": 698,
40     "goal": 5140,
41     "backers": 9,
42     "img": "./assets/camouflage.png"
43   },
44   {
45     "name": "Beep Bapp Boom",
46     "description": "The time to explode is getting shorter and shorter, please be alert!",
47     "pledged": 44,
48     "goal": 18133,
49     "backers": 2,
50     "img": "./assets/beep_bapp_boom.png"
51   },
52   {
53     "name": "Frosthaven",
54     "description": "Euro-inspired dungeon crawling sequel to the 2017 smash hit Gloomhaven",
55     "pledged": 69608,
56     "goal": 500000,
57     "backers": 3193,
58     "img": "./assets/frosthaven.png"
59   },
60   {
61     "name": "Mislight - An Adventure Game With A Small Touch Of Thriller",
62     "description": "Be the Light in a realm of Darkness",
63     "pledged": 1036,
64     "goal": 3099,
65     "backers": 32,
66     "img": "./assets/mislight.png"
67   },
68   {
69     "name": "How to Read Minds 2 Kit: Ellusionist x Peter Turner",
70     "description": "Command Attention With These REAL Mind Reading Techniques That An",
71     "pledged": 147975,
72     "goal": 10000,
73     "backers": 1039,
74     "img": "./assets/how_to_read_minds_2.png"
75   },
76   {
77     "name": "A Wayfarer's Tale",
78     "description": "A Wayfarer's Tale, is a Solo to 4 player game where you explore u",
79     "pledged": 13039,
80     "goal": 1183,
81     "backers": 1446,
82     "img": "./assets/wayfarers_tale.png"
83   },
84   {
85     "name": "Kingdom Death: Monster 1.5",
86     "description": "A cooperative nightmare horror game experience",
87     "pledged": 94139,
88     "goal": 100000,
89     "backers": 9264,
90     "img": "./assets/kingdom_death.png"
91   }
92 ]
```

```
2 const games = [
67   {
68     "name": "How to Read Minds 2 Kit: Ellusionist x Peter Turner",
69     "description": "Command Attention With These REAL Mind Reading Techniques That An",
70     "pledged": 147975,
71     "goal": 10000,
72     "backers": 1039,
73     "img": "./assets/how_to_read_minds_2.png"
74   },
75   {
76     "name": "A Wayfarer's Tale",
77     "description": "A Wayfarer's Tale, is a Solo to 4 player game where you explore u",
78     "pledged": 13039,
79     "goal": 1183,
80     "backers": 1446,
81     "img": "./assets/wayfarers_tale.png"
82   },
83   {
84     "name": "Kingdom Death: Monster 1.5",
85     "description": "A cooperative nightmare horror game experience",
86     "pledged": 94139,
87     "goal": 100000,
88     "backers": 9264,
89     "img": "./assets/kingdom_death.png"
90   }
91 ]
92 ]
```

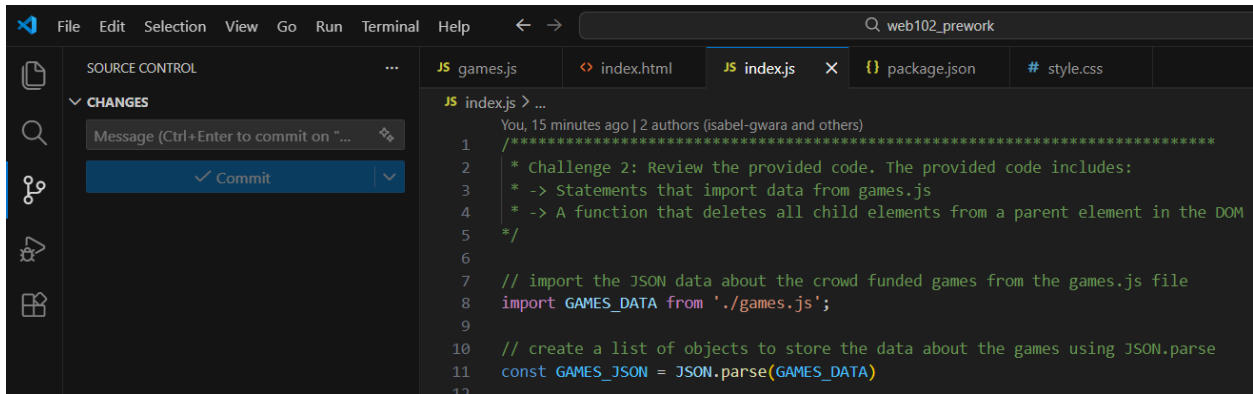
In Secret Key component 2 the output of the following code segment would be The answer to $\{x\} + \{y\}$ is $\{x + y\}$

🔑 **Secret Key component 2:** What would the output of the following code segment be?

```
let x = 10;
let y = 4
console.log("The answer to ${x} + ${y} is ${x + y}");
```

1. The answer to \$10 + \$4 is \$14 (keyword: orange)
2. The answer to \${x} + \${y} is \${x + y} (keyword: seafoam)
3. The answer to 10 + 4 is 14 (keyword: yellow)
4. The answer to 14 is 14 (keyword: lilac)

In Secret Key component 3 the variable that was passed to addGamesToPage in order to add all the games to the page is GAMES_JSON.



```
JS index.js > ...
You, 15 minutes ago | 2 authors (isabel-gwara and others)
1  /*****
2  * Challenge 2: Review the provided code. The provided code includes:
3  * -> Statements that import data from games.js
4  * -> A function that deletes all child elements from a parent element in the DOM
5  */
6
7  // import the JSON data about the crowd funded games from the games.js file
8  import GAMES_DATA from './games.js';
9
10 // create a list of objects to store the data about the games using JSON.parse
11 const GAMES_JSON = JSON.parse(GAMES_DATA)
12
```

What I Learned:

I learned how to use JavaScript to dynamically generate HTML content using loops, functions, and template literals. This challenge helped me understand how data-driven websites render content by manipulating the DOM instead of hardcoding HTML. I also learned from the components that the secret key is **11seafoamGAMES_JSON**.

Challenge 4

Description:

This challenge required calculating summary statistics using `reduce()` and displaying the total number of contributions, total amount pledged, and total number of games at the top of the page.

Code / Work Shown:

In secret key component 1 the value that is now displayed under the Individual Contributions heading is 19187.

Step 1: reduce.

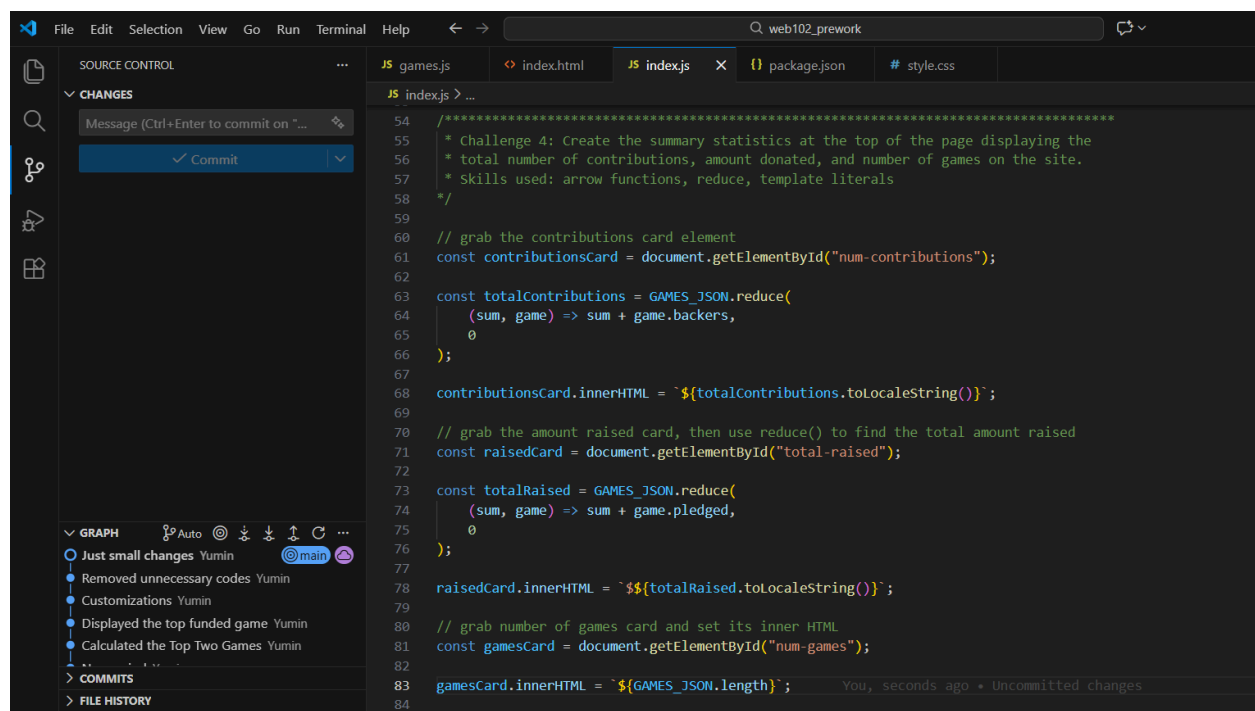
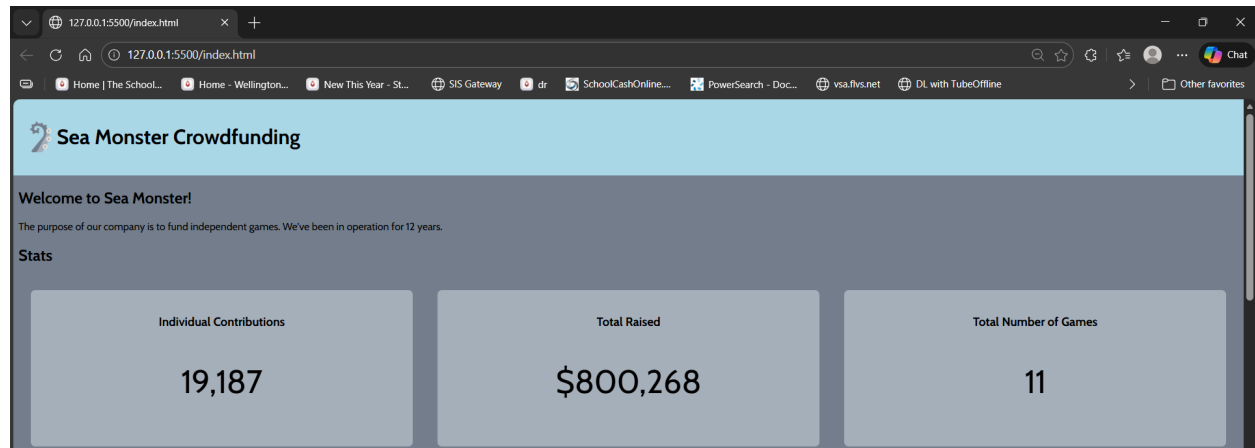
In secret key component 2 the value that is now displayed under the Total Raised heading is 800268.

Step 2: raisedCard.

In secret key component 3 the error this:

The arrow function does not add sum and animal.charAt(0), so it will return only "c" (keyword: BRAIN)

Step 3: gamesCard.



What I Learned:

I learned how to use the `reduce()` function to aggregate data and how to format large numbers using `toLocaleString()` for better readability. I also learned that the secret key from all of the components is **19187800268BRAIN**.

Challenge 5

Description:

This challenge required adding filtering functionality so users could view only funded games, only unfunded games, or all games using buttons and event listeners.

Code / Work Shown:

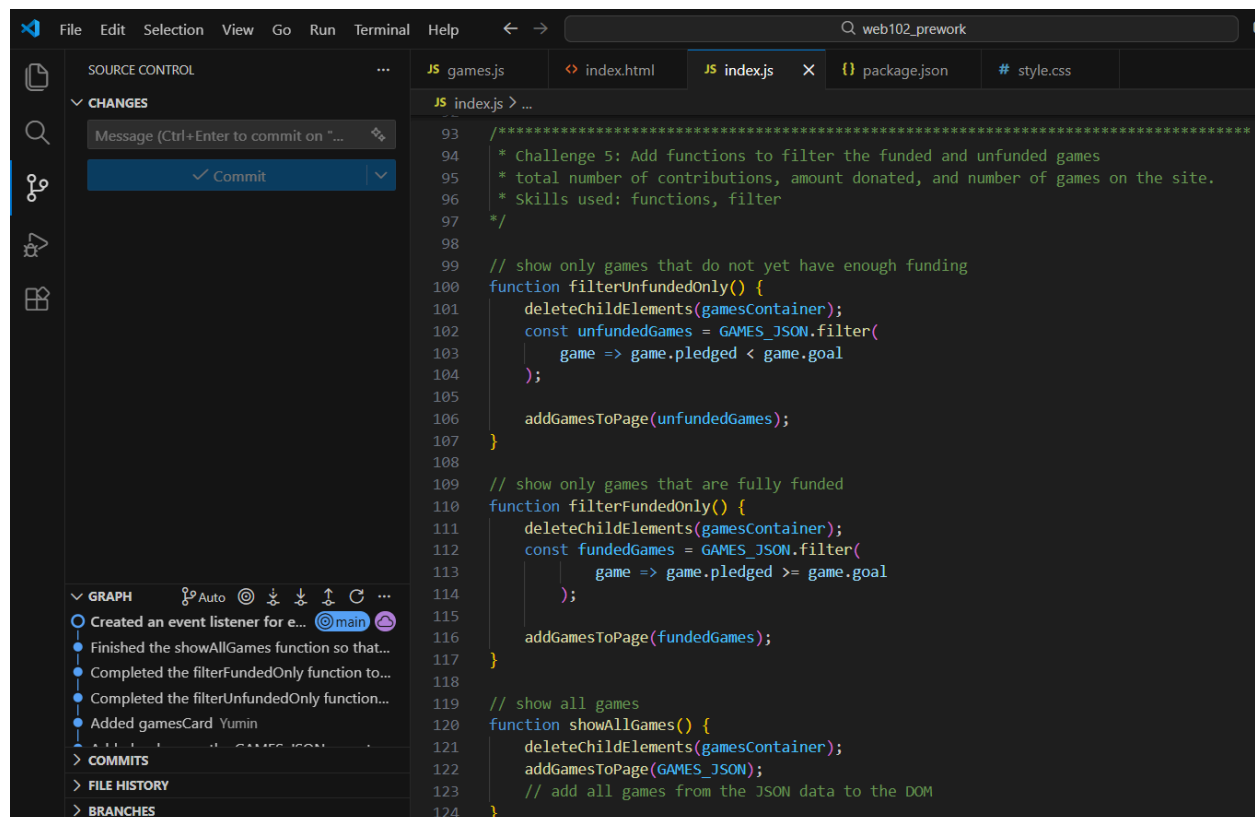
In secret key component 1 the number of games that are in the array returned by filterUnfundedOnly: 7.

In secret key component 2 the number of games that are in the array returned by filterFundedOnly: 4.

In secret key component 3 if the deleteChildElements function was never called within the three event handlers that I wrote then the buttons would always add their list of games to the games already displayed, creating a growing list of games. (keyword: FLANNEL)

In secret key component 4 the type of event each button listen for is click.

Here're my completed filterUnfundedOnly, filterFundedOnly, showAllGames functions, and eventlisteners functions:



```
93  /*
94  * Challenge 5: Add functions to filter the funded and unfunded games
95  * total number of contributions, amount donated, and number of games on the site.
96  * Skills used: functions, filter
97  */
98
99  // show only games that do not yet have enough funding
100 function filterUnfundedOnly() {
101   deleteChildElements(gamesContainer);
102   const unfundedGames = GAMES_JSON.filter(
103     game => game.pledged < game.goal
104   );
105
106   addGamesToPage(unfundedGames);
107 }
108
109 // show only games that are fully funded
110 function filterFundedOnly() {
111   deleteChildElements(gamesContainer);
112   const fundedGames = GAMES_JSON.filter(
113     game => game.pledged >= game.goal
114   );
115
116   addGamesToPage(fundedGames);
117 }
118
119 // show all games
120 function showAllGames() {
121   deleteChildElements(gamesContainer);
122   addGamesToPage(GAMES_JSON);
123   // add all games from the JSON data to the DOM
124 }
```

```
VS Code interface showing a file explorer on the left with 'SOURCE CONTROL' and 'CHANGES' sections. The main editor displays 'JS index.js' with the following code:

function filterFundedOnly() {
  addGamesToPage(fundedGames);
}

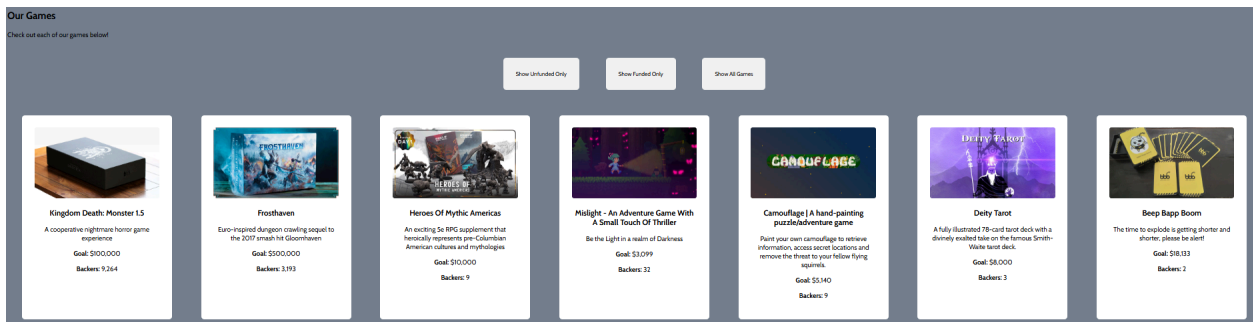
// show all games
function showAllGames() {
  deleteChildElements(gamesContainer);
  addGamesToPage(GAMES_JSON);
  // add all games from the JSON data to the DOM
}

// select each button in the "Our Games" section
const unfundedBtn = document.getElementById("unfunded-btn");
const fundedBtn = document.getElementById("funded-btn");
const allBtn = document.getElementById("all-btn");

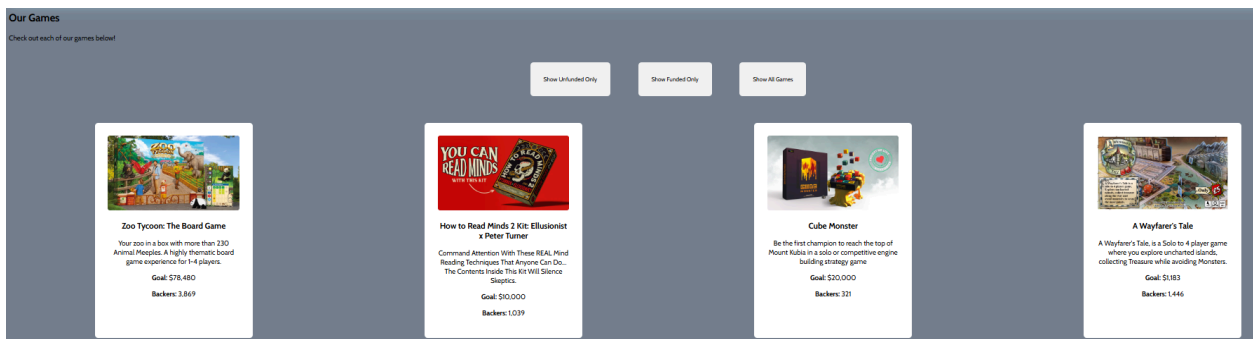
// add event listeners with the correct functions to each button
unfundedBtn.addEventListener("click", filterUnfundedOnly);
fundedBtn.addEventListener("click", filterFundedOnly);
allBtn.addEventListener("click", showAllGames);

isabel-gwara, 3 years ago via PR #1 • Initial commit
```

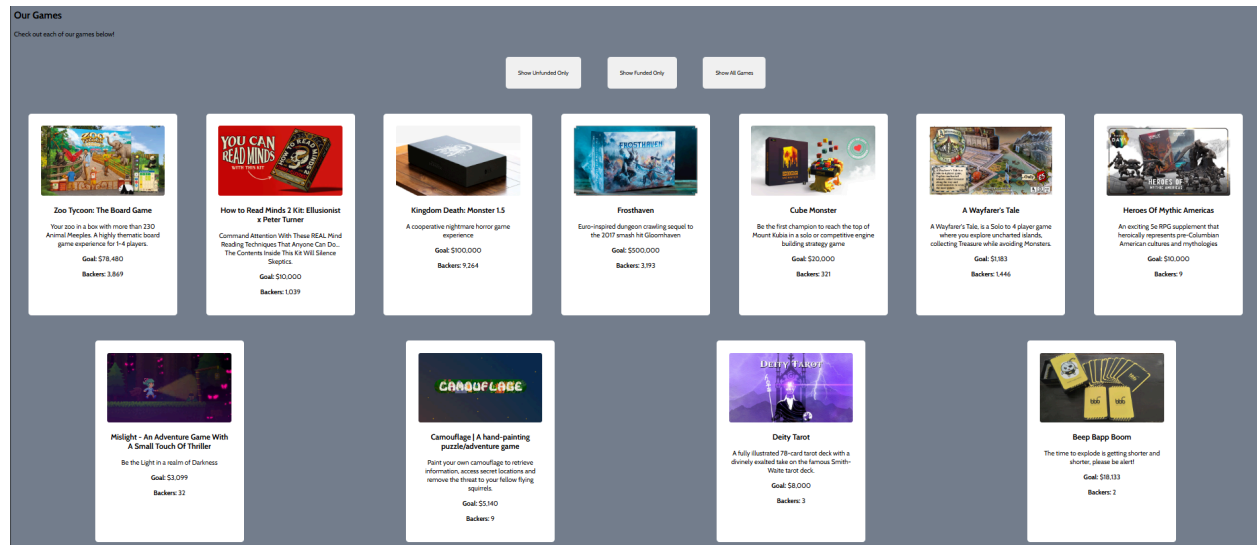
Show unfunded only



Show funded only



Show all games



What I Learned:

I learned how to use `filter()` to create subsets of data and how event listeners allow users to interact with and update the page dynamically. I also learned that the secret key from those four components is **74FLANNELclick**.

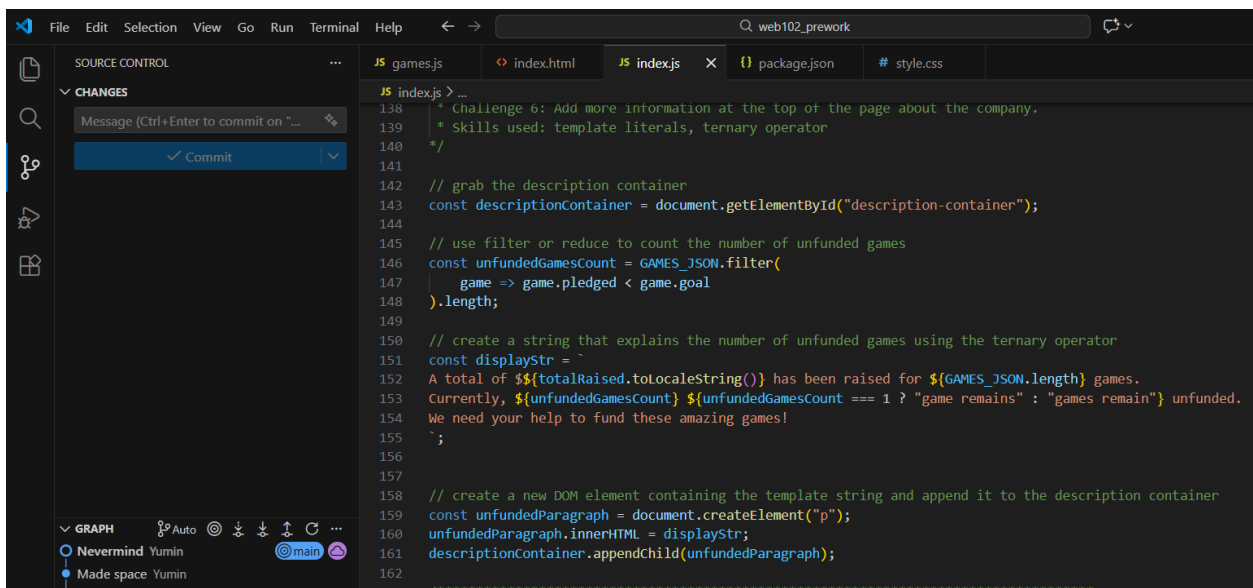
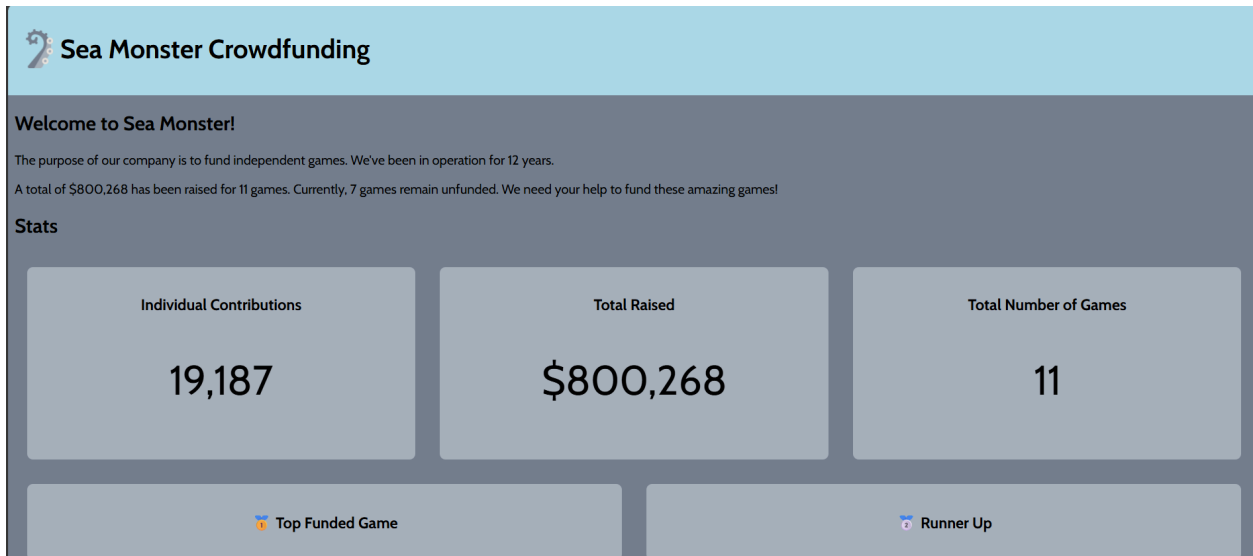
Challenge 6

Description:

In this challenge, I'm supposed to calculate the number of unfunded games using `filter` or `reduce` and display a summary of total funds raised, total games, and unfunded games using a template literal. The ternary operator ensured correct grammar, and the summary was added to the `descriptionContainer` as a new paragraph element.

Code / Work Shown:

Step 1. Use `filter` or `reduce` to sum the number of unfunded games in `GAMES_JSON`.
 Step 2. Use a template string to display how much money has been raised and for how many games, as well as explaining how many games currently remain unfunded.
 Step 3. Create a new paragraph element containing the template string you created and add the element to the `descriptionContainer`.



In Secret Key component 1 the function that adds the correct commas and punctuation to a number, e.g. displaying 100000 as 100,000 is toLocaleString

In Secret Key component 2 the opening tag of the HTML element you appended your newly created element to is <div>.

In Secret Key component 3 the option that would correctly display "Hello {name}" if a user is logged in, and simply "Hello!" if the user is not logged in is option 1.

isLoggedIn ? `Hello \${user.name}` : "Hello!"; (keyword: 1)

In Secret Key component 4 the following code segment returns \$2.00 (keyword: IVY)

What I Learned:

I learned how to use template literals and the ternary operator to generate dynamic,

grammatically correct text. This challenge also reinforced using array methods like filter and reduce combined with DOM manipulation to display meaningful information on a webpage. The secret key for challenge 6 is **toLocaleString<div>1IVY**.

Challenge 7

Description:

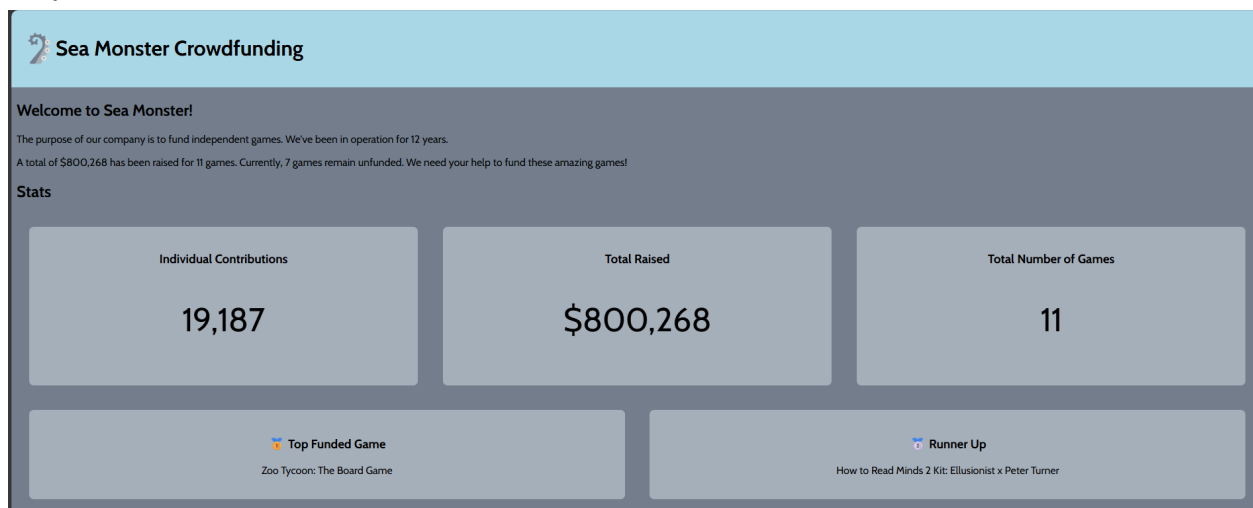
In this challenge, I'm supposed to identify the top two most funded games using destructuring and the spread operator. I then display their names by creating elements and appending them to firstGameContainer and secondGameContainer.

Code / Work Shown:

Step 1: Calculating the Top Games

Step 2: Displaying the Top Games

Step 3: Customizations



```
File Edit Selection View Go Run Terminal Help
web102_prework

SOURCE CONTROL
CHANGES
Message (Ctrl+Enter to commit on "main")
Sync Changes 1 ↑

GRAPH
Outgoing Changes main
Customizations Yumin

JS games.js index.html JS index.js x package.json # style.css
JS index.js > ...
162
163
164 /* Challenge 7: Select & display the top 2 games
165  * Skills used: spread operator, destructuring, template literals, sort
166  */
167
168 const firstGameContainer = document.getElementById("first-game");
169 const secondGameContainer = document.getElementById("second-game");
170
171 const sortedGames = GAMES_JSON.sort((item1, item2) => {
172   return item2.pledged - item1.pledged;
173 });
174
175 // use destructuring and the spread operator to grab the first and second games
176 const [topGame, secondTopGame, ...rest] = sortedGames;
177
178 // create a new element to hold the name of the top pledge game, then append it to the correct element
179 const topGameName = document.createElement("p");
180 topGameName.innerHTML = topGame.name;
181 firstGameContainer.appendChild(topGameName);
182
183 // do the same for the runner up item
184 const secondGameName = document.createElement("p");
185 secondGameName.innerHTML = secondTopGame.name;
186 secondGameContainer.appendChild(secondGameName);
```

In Secret Key component 1 the first word of the most funded game is Zoo.
In Secret Key component 2 the first word of the second most funded game is How.
In Secret Key component 3 the value of ...rest in the following code segment is CEDAR.

What I Learned:

I learned how to extract specific elements from arrays using destructuring and the spread operator, which simplifies handling and displaying key data without mutating the original array. The secret key from the three components is **ZooHowCEDAR**.

END HERE challenge pdf

CODEPATH*ORG

WEB102 Prework

Submitting your challenge!

Checkpoint

Congrats! You've finished all the challenges for the CodePath WEB102 prework! Awesome work completing each of these challenges! You now have a fully functional website that displays information about crowdfunded games and you've learned a ton about JavaScript along the way. You've done a ton of work and now it's time to harvest the fruits of your labor. It's time to turn in your work! The finished website should look something like the screenshot below, not including your customizations, of course.

Submission Instructions

Step 1: Preparing for Submission

 **Important:** If you're taking this course For-Credit at your university, the prework project is optional and does not need to be submitted..

All submissions happen via Github and we require each submission to follow a particular format, including a **clearly documented README** with a list of completed features and a **GIF (or video) website walkthrough**.

Once you have completed these challenges and pushed your code to GitHub with the README and GIF/Video walkthrough included, you'll submit the project.

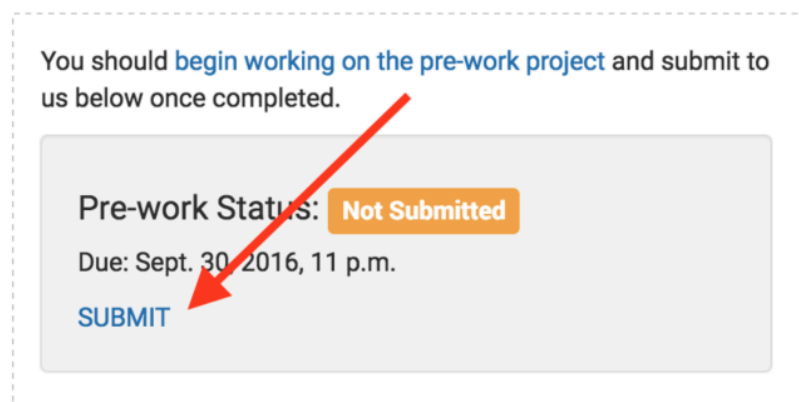
 **Tip:** To make the app walkthrough, we recommend using LiceCap (GIF) or Loom (video). However, you are welcome to use any screen capture tool you wish.

 **Windows Users:** For LiceCap to record GIFs properly in certain versions of Windows, be sure to set Scaling to 100%.

➡➡➡ Step 2: Adding the Pework to Your Application

👉 **Important:** If you're taking this course For-Credit at your university, the prework project is optional and does not need to be submitted..

Once you have completed all required challenges and added a README as described above, your next step is to notify us that you are ready for a prework review. To do this, go to the [application status dashboard](#) and then press the "SUBMIT" button in the prework section:



Make sure that your cohort is correct before submitting. In the page that appears, enter the required information about your project:

➡➡➡ Step 3: Review the Submission Checklist

👉 **Important:** If you're taking this course For-Credit at your university, the prework project is optional and does not need to be submitted..

Please review the following checklist to ensure your submission is valid:

- ☒ Can you view the game stats in the dashboard at the top of the Sea Monster website?
- ☒ Can you use the "Show Unfunded Only" and "Show Funded Only" buttons to view filtered versions of the game list?
- ☒ Did you successfully push your code to GitHub? Can you see the code on GitHub?
- ☒ Did you complete the README.md in the repo on GitHub?
- ☒ Does your README include a GIF walkthrough of the site's functionality?
- ☒ Did you visit your application dashboard and submit using the prework form?

If the answer to all of these is YES then, 🎉 **Congratulations, you've successfully completed the required portion of the prework!** After you've completed these steps, you'll hear from us soon with more information about the rest of the selection process.